

```

#include <stdio.h>

// Arrays to store block positions and stacks
int place_hol[30]; // Indicates the current position of each block
int stack[30][30]; // Represents the stacks of blocks
int on_top[30]; // Tracks the top position in each stack

// Function to reset block positions
void init_place(int a)
{
    int block, id = place_hol[a];

    // Move blocks above 'a' back to their initial positions
    while (stack[id][on_top[id]] != a)
    {
        block = stack[id][on_top[id]--];
        place_hol[block] = block;
        stack[block][++on_top[block]] = block;
    }
}

// Function to pile blocks 'a' and 'b'
int temp[30]; // Temporary array to store blocks when performing pile operation
void pile(int a, int b)
{
    int topt = -1, id = place_hol[a], ID = place_hol[b];

    // Move blocks above 'a' to temporary storage
    while (stack[id][on_top[id]] != a) temp[++topt] = stack[id][on_top[id]--];

    place_hol[a] = ID; // Update block 'a' position
    stack[ID][++on_top[ID]] = a; // Place 'a' onto the stack of 'b'
    on_top[id]--; // Update top position for block 'a's original position

    // Move blocks from temporary storage onto the stack of 'b'
    while (topt >= 0)
    {
        place_hol[temp[topt]] = ID;
        stack[ID][++on_top[ID]] = temp[topt--];
    }
}

int main()
{
    int n, a, b;
    char oper[5], type[5];

    while (scanf("%d", &n) == 1)
    {
        // Initialize block positions and stacks
        for (int i = 0; i < n; ++i)
        {
            stack[i][0] = i; // Each block is initially in its own stack
            place_hol[i] = i; // The block is initially in its own position
            on_top[i] = 0; // There's only one block in each stack
        }

        // Commands to move and pile blocks
        while (scanf("%s", oper) && oper[0] != 'q')
        {

```

```

scanf("%d%s%d", &a, type, &b);

// Checks if blocks are in the same position or the command is 'q'
if (place_hol[a] == place_hol[b]) continue;

// Moves block 'a' to its initial position if the command is 'm'
if (oper[0] == 'm') init_place(a);
// Moves block 'b' to its initial position if the type is 'n'

if (type[1] == 'n') init_place(b);

// Pile blocks 'a' and 'b'
pile(a, b);
}

// Print the current arrangement of blocks
for (int i = 0; i < n; ++i)
{
    printf("%d:", i);

    int now = 0;

    while (now <= on_top[i]) printf(" %d", stack[i][now++]);

    printf("\n");
}
}

return 0;
}

```